RAG-POWERED ASSISTANT FOR INFRASTRUCTURE AS CODE GENERATION

BITS SEZG628T: Dissertation (Mid-Semester Report)

by

KOLLA NAGA MANASA
2024TM93519

Dissertation work carried out at

Caprus IT Private Limited, Hyderabad

Submitted in partial fulfilment of **M.Tech (Software Engineering)** degree programme

Under the Supervision of

Karthik Reddy Charla

Caprus IT Private Limited, Hyderabad



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE

PILANI (RAJASTHAN)

September 2026

ABSTRACT

Writing Infrastructure as Code (IaC) by hand is slow and error-prone, and large language models (LLMs) can draft it but often ignore organisational conventions that a plain-language request never states. This dissertation builds and evaluates a Retrieval-Augmented Generation (RAG) assistant that turns natural-language requirements into Terraform for Google Cloud, grounded in the public provider documentation and a set of organisational conventions, and checks every output with terraform validate and plan, Open Policy Agent policies and functional-intent checks.

The prototype runs end to end on Vertex AI with Gemini. It is evaluated on 35 tasks, three arms (RAG, a no-retrieval baseline, and a control that pastes the conventions into the prompt) and five samples per task, 525 samples in all. Technical correctness is near ceiling for every arm (functional pass 95% to 99%), so retrieval does not measurably improve it. The arms differ on organisational policy: the overall pass rate is 57.0% for the baseline, 78.8% for RAG and 95.2% for the control. RAG beats the baseline on 10 tasks and loses on none. However, the control beats RAG on 10 tasks and loses on 3. The stored records suggest why: the conventions document was retrieved in only 12% of RAG samples, and the policy failures concentrate where it was absent.

Retrieval-count and per-document-cap ablations and a near-greedy run agree with these findings. The remaining work is a chunk-size ablation, replacing the hand-written conventions with reviewed internal templates, deeper error analysis and a user study.

Signature of the Student	**Signature of the Supervisor**
Name: Kolla Naga Manasa	Name: Karthik Reddy Charla
Date:	Date:
Place: Hyderabad	Place: Hyderabad

Contents

ABSTRACT 2

1. INTRODUCTION 7

1.1 Background and motivation 7

1.2 Problem statement 7

1.3 Objectives and current status 7

1.4 Scope 8

1.5 Organisation of this report 8

2. LITERATURE REVIEW 9

2.1 Retrieval-Augmented Generation 9

2.2 LLMs for infrastructure code and their benchmarks 9

2.3 Error taxonomies and knowledge injection 9

2.4 Verifier-based evaluation and policy compliance 10

2.5 Risks introduced by retrieval 10

2.6 Gap addressed by this work 11

3. SYSTEM DESIGN 12

3.1 Design principles 12

3.2 Architecture 12

3.3 Technology choices 13

3.4 Knowledge base 14

3.5 Chunking and retrieval 15

3.6 Generation 15

3.7 Validation stack 15

3.8 Safety 16

3.9 UML Design Views 16

4. IMPLEMENTATION 25

4.1 Code base 25

4.2 Reproducibility 25
4.3 Local development mode 25
4.4 Engineering problems found and fixed 25
5. EVALUATION METHODOLOGY 27
5.1 Research questions 27
5.2 Task set 27
5.3 Experimental arms, sampling and metrics 28
5.4 Retrieval quality and the error taxonomy 29
5.5 Statistical treatment 29
5.6 Threats to validity 30
6. RESULTS 31
6.1 The run 31
6.2 Headline metrics 31
6.3 Paired comparison across tasks 32
6.4 Error analysis 34
6.5 Retrieval quality and outcome 35
6.6 By category and difficulty 36
6.7 Requirements that conflict with organisational policy 38
6.8 Retrieval ablations 38
6.9 Near-greedy run (temperature 0) 39
7. DISCUSSION 40
8. REMAINING WORK AND PLAN 42
8.1 Remaining technical work 42
8.2 Proposed user evaluation 42
8.3 Future Plan 43
8.4 Risks 43
9. CONCLUSION 44

ABBREVIATIONS 45

List of Figures

Figure 1: System architecture: offline knowledge-base construction, online generation for the two experimental arms, and identical validation for both. 13
Figure 2: Use-case diagram of the RAG-IaC assistant. 17
Figure 3: Component diagram showing the main components and their dependencies. 18
Figure 4: Class diagram of the pipeline and the generators. 19
Figure 5: Class diagram of the retrieval subsystem. 20
Figure 6: Class diagram of the validation and classification model. 21
Figure 7: Sequence diagram of a RAG request and its validation. 22
Figure 8: Activity diagram of the benchmark evaluation. 23
Figure 9: Deployment diagram of the prototype. 24
Figure 10: Pass rates by arm with 95% confidence intervals. 32
Figure 11: Share of samples containing each error category. 34
Figure 12: Overall pass rate by task category. 37

List of Tables

Table 1: Objectives and their status. 8
Table 2: Technology choices and rationale. 14
Table 3: The four validation levels. 16
Table 4: Defects found in the tooling and their remedies. 26
Table 5: Task set composition. 27
Table 6: Metrics (validate/plan pass rate, OPA violation rate and pass@k were specified by the supervisor; the remainder were added). 29
Table 7: Headline metrics. Proportions show the 95% Wilson interval in brackets. 31
Table 8: Tasks with the largest per-task differences between RAG and the baseline (at least 40 percentage points). 33
Table 9: Tasks with the largest per-task differences between RAG and the conventions-in-prompt control (at least 40 percentage points). 33

Table 10: Samples containing each error category. A sample can contain several. 34
Table 11: The most frequent validator and intent diagnostics (policy findings are shown in the previous table). 35
Table 12: RAG-arm outcomes by whether documentation for every needed resource was retrieved. 35
Table 13: Overall pass rate by task category. 36
Table 14: Overall pass rate by difficulty level. 37
Table 15: The two tasks that explicitly request policy-forbidden configurations. 38
Table 16: RAG arm under different retrieval settings. Proportions show the 95% Wilson interval in brackets. 38
Table 17: Headline metrics at temperature 0, one sample per task. 39
Table 18: Proposed user-evaluation design. 42
Table 19: Plan of work and status. 43
Table 20: Abbreviations used in this report. 45

1. INTRODUCTION

1.1 Background and motivation

Infrastructure as Code (IaC) is now the standard way of provisioning and managing cloud resources: the desired infrastructure is described declaratively in version-controlled files, and a tool such as Terraform reconciles the real environment with that description. Writing correct IaC is nevertheless difficult. Provider schemas are large and change frequently (the Google Cloud provider used in this work was at major version 8 at the time of writing), resources depend on one another in ways that the documentation only describes indirectly, and every organisation layers its own conventions and security policies on top.

Large language models (LLMs) can turn a plain-English requirement into a plausible Terraform configuration, but plausibility is not correctness. A model's knowledge is frozen at training time, so it tends to produce arguments that have been renamed or removed, omits arguments that are now required, and knows nothing about the conventions of the organisation it is being used in. Retrieval-Augmented Generation (RAG) addresses exactly this weakness by placing retrieved, current documentation in the model's context at generation time instead of relying on what the model memorised.

The problem is directly relevant to the author's employer, Caprus IT, where delivery work provisions infrastructure largely through Terraform and where misconfiguration and the slow onboarding of new engineers continue to cost time. An assistant that converts plain-language requirements into validated, policy-compliant IaC would reduce authoring effort and lower the rate of misconfiguration-related incidents.

1.2 Problem statement

Given a natural-language infrastructure requirement, generate Terraform for Google Cloud Platform (GCP) that (a) is well-formed and consistent with the provider's schema, (b) actually implements what was asked, and (c) complies with organisational policy. The central research question is whether retrieval over provider documentation and organisational conventions improves these outcomes compared with the same model used without retrieval, and, where it does not, what kinds of errors remain.

1.3 Objectives and current status

The objectives set out in the approved outline, and where each stands at the mid-semester point, are:

#	Objective (from the outline)	Status at mid-semester
O1	Study the state of the art in LLM-based code generation and RAG for IaC	Done for this report (Section 2); continues through the final report
O2	Identify common LLM error categories in IaC generation and the knowledge gaps retrieval must close	Taxonomy implemented and applied to every benchmark sample (Sections 5.4, 6)
O3	Design a RAG architecture (ingestion, chunking, embedding, vector storage, retrieval, generation)	Done (Section 3)
O4	Build a working prototype that generates IaC from natural-language prompts	Done: runs end to end on real GCP services (Sections 3, 4)
O5	Validate output with automated tooling and benchmark against a non-RAG baseline	Tooling done; full benchmark, control arm, retrieval ablations and a near-greedy run reported (Section 6); chunk-size ablation not run
O6	User evaluation of usability and developer trust	Designed and proposed (Section 8.2); to be run in Phase 6

Table 1: Objectives and their status.

1.4 Scope

The work designs, implements and evaluates a RAG-based assistant that accepts natural-language infrastructure requirements and produces Terraform for Google Cloud, grounded in a curated knowledge base of provider documentation and organisational conventions. Production deployment, multi-cloud coverage beyond GCP, and continuous-integration rollout are outside the scope of this dissertation and are noted as future work. The assistant is never used to apply infrastructure: generated code is only validated and planned.

1.5 Organisation of this report

Section 2 reviews related work. Section 3 describes the system design and the reasoning behind each technology choice. Section 4 covers implementation, reproducibility and the engineering problems found and fixed along the way. Section 5 sets out the evaluation methodology, including the metrics, which were fixed before any results were collected. Sections 6 and 7 present and discuss the results of the first full benchmark. Section 8 describes the remaining work, including the proposed user evaluation, and Section 9 concludes.

2. LITERATURE REVIEW

2.1 Retrieval-Augmented Generation

Lewis et al. [1] introduced Retrieval-Augmented Generation as a way of combining a generator's parametric memory (knowledge stored in model weights) with a non-parametric memory: a dense index of documents that is queried at inference time and whose results condition the output. Two properties make the approach attractive for infrastructure code. The knowledge source can be updated without retraining the model, which matters when provider schemas change every few weeks, and the retrieved passages give generated output a traceable provenance. Khan et al. [8] report the practical side of building such systems, describing the end-to-end pipeline from data collection and preprocessing through retrieval indexing to response generation and the technical problems that arise at each stage. The present system follows the same staged decomposition.

2.2 LLMs for infrastructure code and their benchmarks

IaC-Eval [2] is, to our knowledge, the first dedicated benchmark for LLM-generated Terraform. It contains 458 human-curated scenarios and goes beyond syntax by checking whether a generated configuration satisfies an explicit intent specification. Its released tooling targets AWS, with other providers left as future work. This design, in which validity and intent are separate questions, is the model for the intent checks used in this work. Multi-IaC-Eval [5] extends evaluation across AWS CloudFormation, Terraform and CDK and reports that modern LLMs achieve success rates above 95% in generating syntactically valid IaC across formats, while semantic alignment and complex infrastructure patterns remain difficult. Taken together, these results show that syntactic validity is close to saturated for strong models and is therefore an insufficient measure of quality on its own.

2.3 Error taxonomies and knowledge injection

The closest prior work to this dissertation is that of Nekrasov et al. [3], who propose an error taxonomy for LLM-generated Terraform and enhance IaC-Eval with cloud emulation and automated error analysis. They compare knowledge-injection strategies that progress from naive RAG to Graph RAG with semantic enrichment and inter-resource dependency modelling. Their reported baseline LLM success is 27.1% overall; with knowledge injection, technical validation success rises to 75.3% and overall success to 62.6%. They identify a "Correctness-Congruence Gap": models

become proficient coders but remain limited as architects in satisfying nuanced user intent. This is the reason the present evaluation reports well-formedness and intent satisfaction as separate quantities rather than a single score.

2.4 Verifier-based evaluation and policy compliance

Jouini [4] evaluates seven agentic strategies for Terraform generation on a 186-task benchmark using three verifier stages: terraform validate, terraform plan and OPA policy evaluation. Active retrieval raised the pass@1 of one small open model from 14.0% to 45.7% by reducing validation failures, and iterative refinement with verifier feedback reached 62.9% and 84.4% pass@1 for two further models. A finding that bears directly on this design is that 79% of the OPA failures remaining after refinement were information-gap failures that could be resolved when the policy text was visible to the model. That supports placing organisational conventions in the retrieval corpus, which is what this work does, and it motivates the same three verifier stages here.

Vargas et al. [6] benchmark seven proprietary and open models on secure AWS Terraform generation across 17 scenarios using Checkov and Trivy. They conclude that code validity and security are largely orthogonal properties and that prompt engineering alone is insufficient, so automated scanning remains necessary. The consequence for the present study is that technical correctness and policy compliance are reported as distinct dimensions (Section 5.3) so that the effect of retrieval on each can be seen separately.

2.5 Risks introduced by retrieval

Retrieval also adds an attack surface. Ye et al. [7] describe ImportSnare, an attack in which poisoned documentation steers a retrieval-augmented code generator toward malicious dependencies; they report attack success rates above 50% for some popular libraries and success even at a poisoning ratio as low as 0.01%. Although their setting is package recommendation rather than infrastructure, the lesson transfers: whatever enters the knowledge base can shape the generated code. This work therefore restricts the corpus to a pinned release of the official provider documentation plus a small set of reviewed organisational conventions, and accepts no user-contributed content (Section 3.4).

2.6 Gap addressed by this work

In the literature reviewed for this report, evaluation of LLM-generated Terraform is concentrated on AWS, and we did not find a GCP-specific benchmark. Existing studies also tend to report a single success figure; the combination of separate technical, intent and organisational-policy measurements, a per-sample error taxonomy, and a paired retrieval-versus-no-retrieval comparison on the same model and prompt is the contribution attempted here, for Google Cloud. The task set built for this purpose (Section 5.2) follows IaC-Eval's intent-specification approach but is a new, GCP-specific set of 35 tasks, so its results are not directly comparable to figures published for the AWS benchmarks.

3. SYSTEM DESIGN

3.1 Design principles

Five principles guided the design, each chosen so that the eventual results can be trusted.

1. **A controlled comparison.** The two experimental arms use the same model, prompt template, target provider version and sampling settings. They differ in one respect only: whether retrieved context is supplied. Any difference in outcome can then be attributed to retrieval.
2. **Everything is measurable.** Each stage exposes its output so that failures can be attributed to retrieval, generation or validation rather than to the system as a whole.
3. **Safety by construction.** Generated code is only ever validated and planned, never applied, and planning runs against a dedicated experimentation project.
4. **Reproducibility.** Every run records the model, temperature, index size, tool versions, source-document release and hashes of the task file and policies, and stores every generated configuration.
5. **Provenance.** The knowledge base contains only public provider documentation, pinned to a release, and reviewed organisational conventions; no customer or production configuration is used.

3.2 Architecture

Figure 1 shows the pipeline. An offline stage builds the knowledge base: documents are cleaned, split into chunks, embedded and stored in a vector index. At run time a requirement is either passed through the retriever (RAG arm) or straight to the model (baseline arm); the generated Terraform then passes through the same validation and evaluation stages regardless of arm.

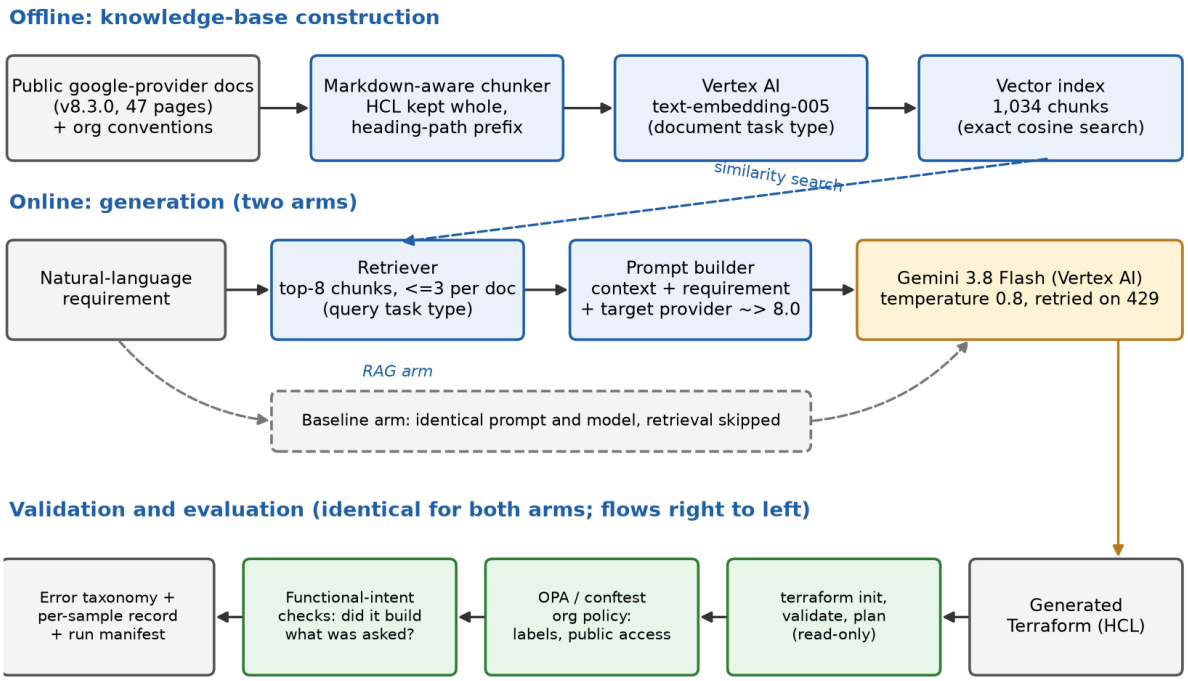


Figure 1: System architecture: offline knowledge-base construction, online generation for the two experimental arms, and identical validation for both.

3.3 Technology choices

Table 2 lists the choices that the supervisor asked to be settled and stated at this stage, with the reasoning for each.

Concern	Choice	Rationale
Cloud and IaC tool	Google Cloud Platform; Terraform with the `google` provider (`~> 8.0`)	Terraform is the delivery team's tool; GCP is the cloud used for all experimentation.
Region	`asia-south1` (Mumbai) for storage and embeddings	Google Cloud has no region in Hyderabad. Of its two Indian regions, Mumbai is roughly 600 km away versus roughly 1,350 km for Delhi, and no cost reason to prefer Delhi was identified, so proximity decided it.
Generation model	Gemini 3.8 Flash on Vertex AI, global endpoint	The current general-availability Flash model. `gemini-2.0-flash` no longer exists on Vertex AI and `gemini-2.5-flash` is scheduled to shut down on 20 October 2026, inside this project's timeline. The global endpoint avoids losing access to newer models that are not yet regional.
Embeddings	Vertex AI `text-embedding-005` (768 dimensions)	Supports separate document and query task types, which the model is trained to expect for asymmetric retrieval.
Vector store	In-memory exact cosine search, persisted to disk	See Section 3.3.1.
Policy engine	Open Policy Agent via Conftest 0.70.0 (OPA 1.20.2)	Policy-as-code checks named in the outline; Rego rules are versioned with the code.
Validation	Terraform 1.16.2: init, validate, plan	Standard tool-level verification; plan is read-only.

Table 2: Technology choices and rationale.

3.3.1 Vector store decision

The knowledge base is about 1,000 chunks (roughly 1 MB of text). At that size an exact cosine search over an in-memory matrix returns in well under a millisecond and needs no infrastructure. Vertex AI Vector Search is an approximate-nearest-neighbour service designed for very large corpora and is billed while an index endpoint is deployed, so at this scale it would add cost and operational overhead without improving retrieval quality. The vector store sits behind a small interface and a Vertex AI Vector Search implementation is already written, so moving to it is a configuration change if the corpus grows substantially. The decision is therefore an in-memory store for the dissertation, with a documented migration path.

3.4 Knowledge base

The knowledge base combines two sources. The first is 47 pages of public documentation for the `google` Terraform provider, taken from release v8.3.0 of the official repository (MPL-2.0 licence). The pages cover the resources most often needed for common infrastructure: storage, networking, compute, databases, Kubernetes, serverless, messaging, analytics, security, and platform services such as Artifact Registry and DNS. Each page is cleaned by removing generated front matter and the import and timeout sections, which add noise without helping generation; a comment recording the source URL, release and licence is added to the top.

The second source is a set of four short documents written for this project that state organisational conventions in the form a practitioner would document them: the default region, mandatory `environment` labels, no public exposure unless explicitly requested, deletion protection and backups for production data stores, and regional over zonal configurations for production. They stand in for the sanitised internal templates the supervisor asked to be used; replacing them with reviewed internal templates is a remaining task (Section 8).

Because retrieved text shapes generated code, the corpus is treated as a trust boundary (Section 2.5). It is fetched over HTTPS from the official repository at a pinned tag, is regenerated by a script rather than edited by hand, and contains no customer, production or credential data.

3.5 Chunking and retrieval

Provider documentation is long: a single resource page can run to tens of thousands of characters, and its most valuable content is a fenced HCL example whose lines must stay together. A plain paragraph splitter would cut examples at blank lines. The chunker therefore keeps fenced code blocks whole, splits oversized blocks only at line boundaries and re-fences each piece, and prefixes every chunk with its heading path (for example, `[google_storage_bucket > Argument Reference]`) so that a chunk from the middle of a long argument list still identifies the resource it describes. The corpus yields 1,034 chunks with a 1,500-character target.

Queries are embedded with the retrieval-query task type and chunks with the retrieval-document type. The retriever returns the eight most similar chunks with at most three from any single document. The cap was added after inspection showed that a long page could fill every slot: for a requirement asking for a VPC with subnets and Cloud NAT, the top eight results came entirely from the NAT and subnetwork pages and none described the network resource itself. Even with the cap, retrieval remains imperfect, which is why retrieval quality is measured explicitly (Section 5.4) rather than assumed.

3.6 Generation

The system prompt tells the model it is generating Terraform for GCP, to treat retrieved documentation as authoritative in preference to its own memory, to say so in a comment when the retrieved context does not cover something rather than guessing, to include provider and required-provider blocks, and to follow secure defaults. The baseline prompt is identical except that it omits the retrieval instructions and states that no reference material is provided. Both prompts name the target provider version constraint, because otherwise a model that recalls an older provider from memory would be validated against a different schema from the one the retrieved documents describe, confounding the comparison.

Model replies are post-processed to extract the Terraform: models often wrap code in a Markdown fence and sometimes add commentary, and newer models can return several content parts. Transient quota errors are retried with exponential backoff and jitter.

3.7 Validation stack

Generated code is checked at three levels, each answering a different question (Table 3).

Level	Tool	Question answered	Example failure it catches
Well-formed	`terraform init` and `validate`	Is the code syntactically valid and consistent with the provider schema?	An argument that does not exist in provider v8
Plannable	`terraform plan` (read-only)	Can Terraform build an execution plan against the real API?	An invalid combination of values rejected by the provider
Policy-compliant	OPA through Conftest	Does it obey organisational policy?	A bucket with no `environment` label; an `allUsers` IAM grant; a firewall open to 0.0.0.0/0
Intent-satisfying	Task-specific intent checks	Did it build what was asked?	A valid, compliant configuration for the wrong resource, or with the wrong retention period

Table 3: The four validation levels.

Three details matter for the credibility of the measurements. First, plan failures caused by the environment (missing credentials) or by a variable declared without a default are recorded but not counted against the model, since they say nothing about the code. Second, when the policy engine cannot parse a configuration at all, that is recorded as a syntax failure and not as a policy violation. Third, the policy rules were verified against known-violating and known-compliant configurations, because policy code can fail silently (Section 4.4).

3.8 Safety

The system never runs `terraform apply`. Plan runs only against a dedicated experimentation project and is read-only. Credentials are Application Default Credentials held locally, and no secrets are stored in the repository.

3.9 UML Design Views

This section presents the design as UML diagrams. They were drawn from the code base, so the classes, methods and call sequences shown are those implemented, not an idealised design.

3.9.1 Use-case view

Two people use the system. A developer requests Terraform for a natural-language requirement and reviews the validation report; this use case includes retrieving documentation context (in the RAG arm), generating Terraform with the LLM, and validating it with terraform and the policy checks. A researcher builds the knowledge-base index, runs the benchmark over the three arms, and summarises and compares the results. Re-running samples lost to infrastructure failure extends the benchmark.

Vertex AI, Terraform and Conftest are external systems on which the assistant depends.

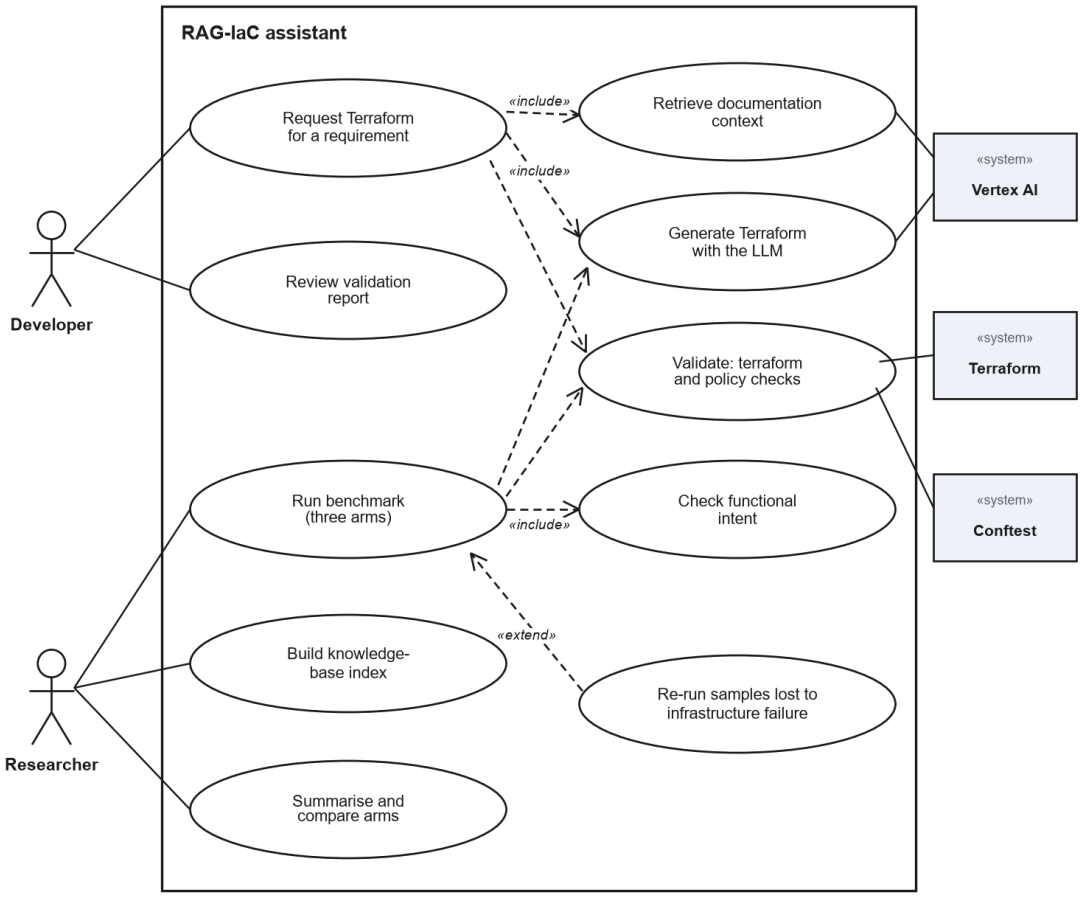


Figure 2: Use-case diagram of the RAG-IaC assistant.

3.9.2 Component view

The Pipeline class is a facade over ingestion, retrieval, generation and validation. The Streamlit UI and the command-line scripts call it, and the evaluation package calls it to run the benchmark. Embeddings and the vector store sit behind interfaces so that an implementation can be replaced without changing their callers. External dependencies are Vertex AI for embeddings and Gemini, the terraform and conftest programs run as local processes, and local files for the knowledge base, the vector index and the results. The Vertex AI Vector Search store is implemented but not provisioned (Section 3.3).

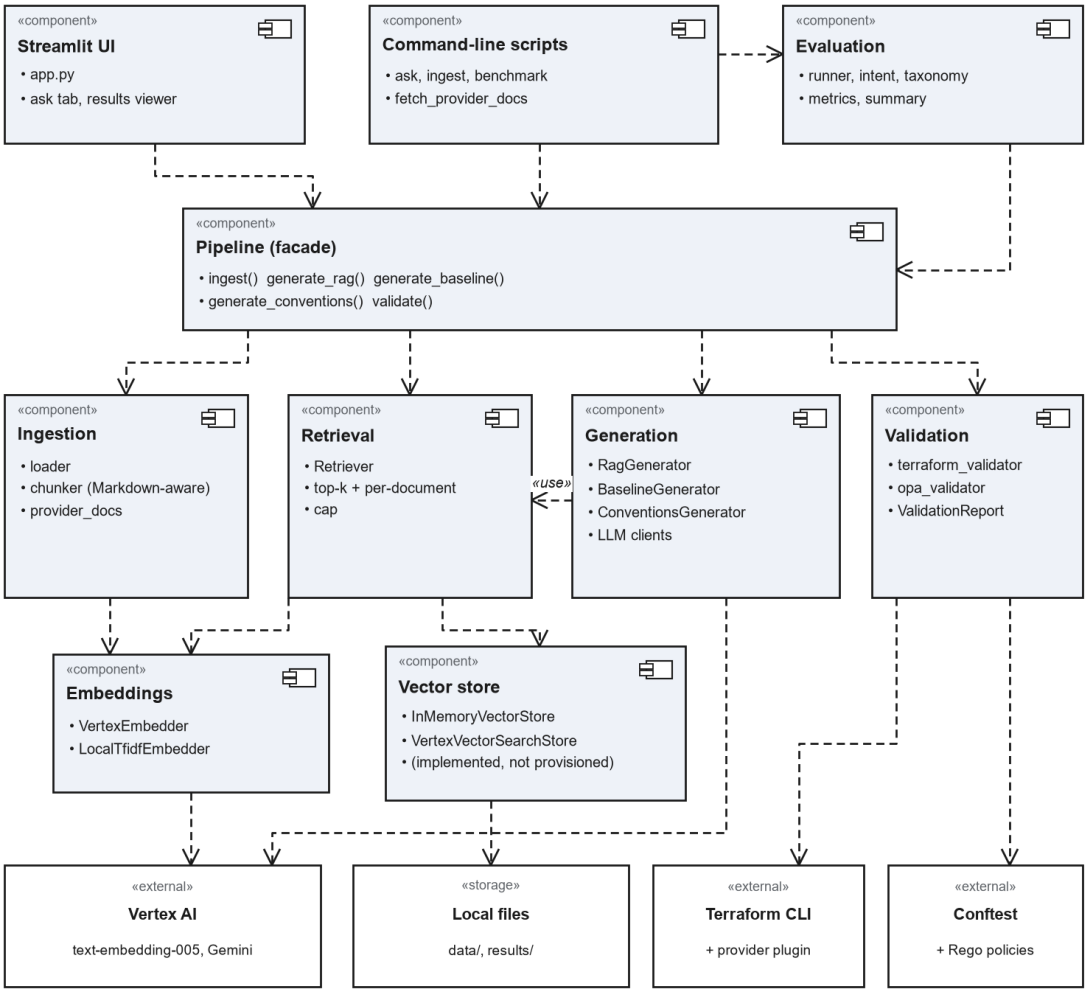


Figure 3: Component diagram showing the main components and their dependencies.

3.9.3 Class views

Figure 4 shows the classes that generate Terraform. Pipeline holds the embedder, vector store, retriever and the three generators, and exposes ingest, the three generate methods and validate. All three generators use one LLM interface and differ only in the context they supply: retrieved chunks, all convention documents, or none. LLM is a structural interface (a Python Protocol) with a Gemini implementation and a rule-based implementation used only for offline development.

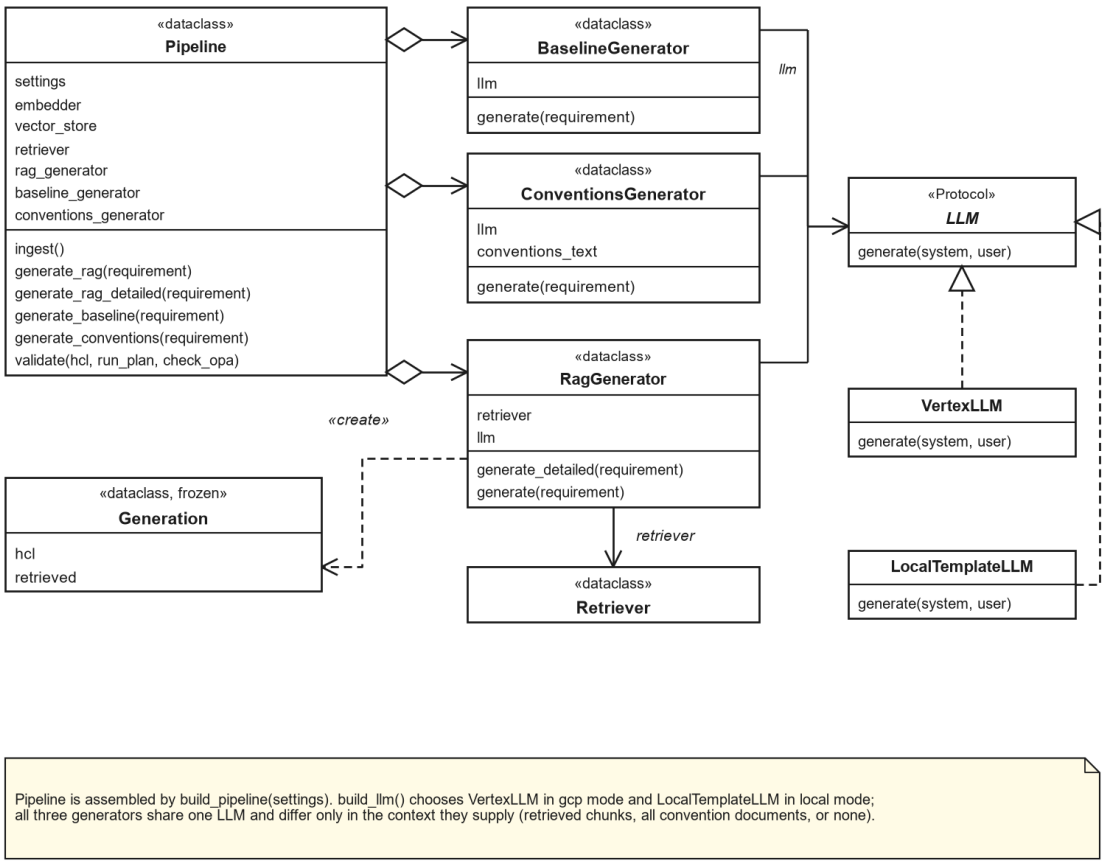


Figure 4: Class diagram of the pipeline and the generators.

Figure 5 shows the retrieval subsystem. The Retriever depends on the Embedder and VectorStore interfaces. The experiments use the Vertex AI embedder and the in-memory store; the TF-IDF embedder supports offline development and the Vertex AI Vector Search store supports a larger corpus. Search returns scored chunks, each holding one chunk of a source document.

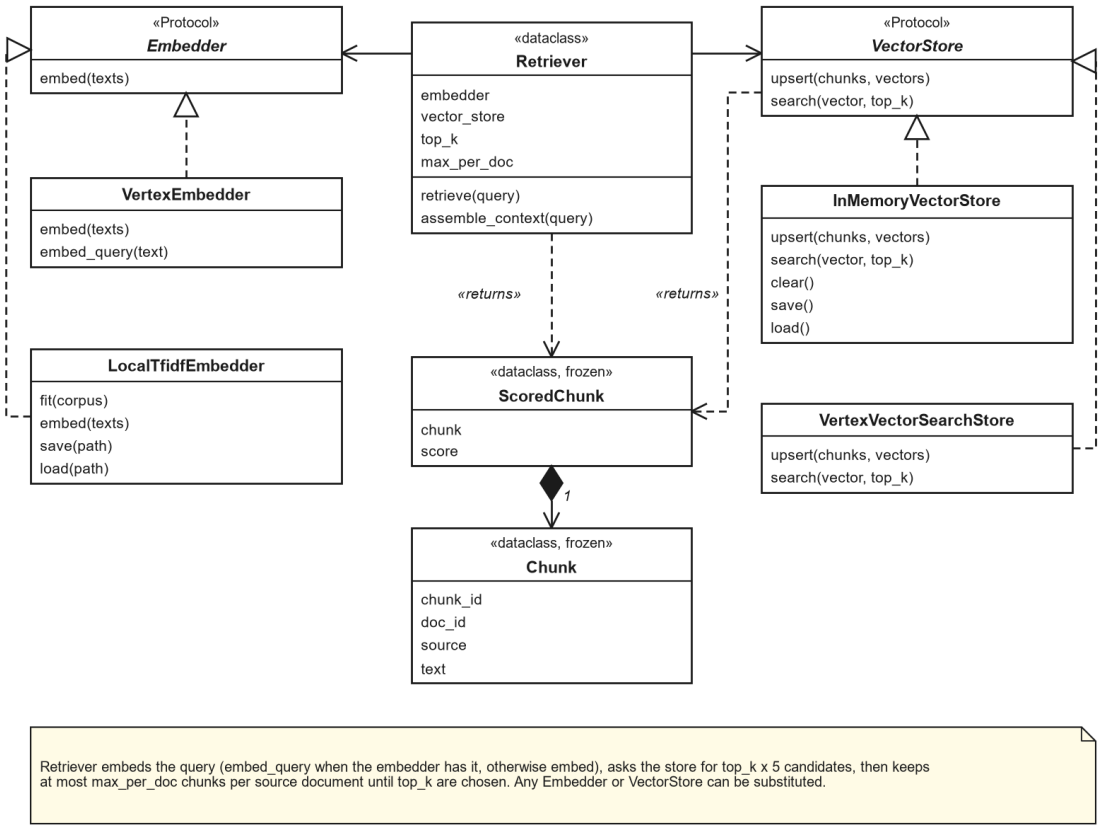


Figure 5: Class diagram of the retrieval subsystem.

Figure 6 shows the validation and classification data model. A ValidationReport holds one result per terraform stage and an optional policy result. The taxonomy maps the validator outputs and the intent result onto error categories. The policy result records whether conftest could evaluate the input, so that HCL that cannot be parsed is counted as a syntax error and not as a policy violation.

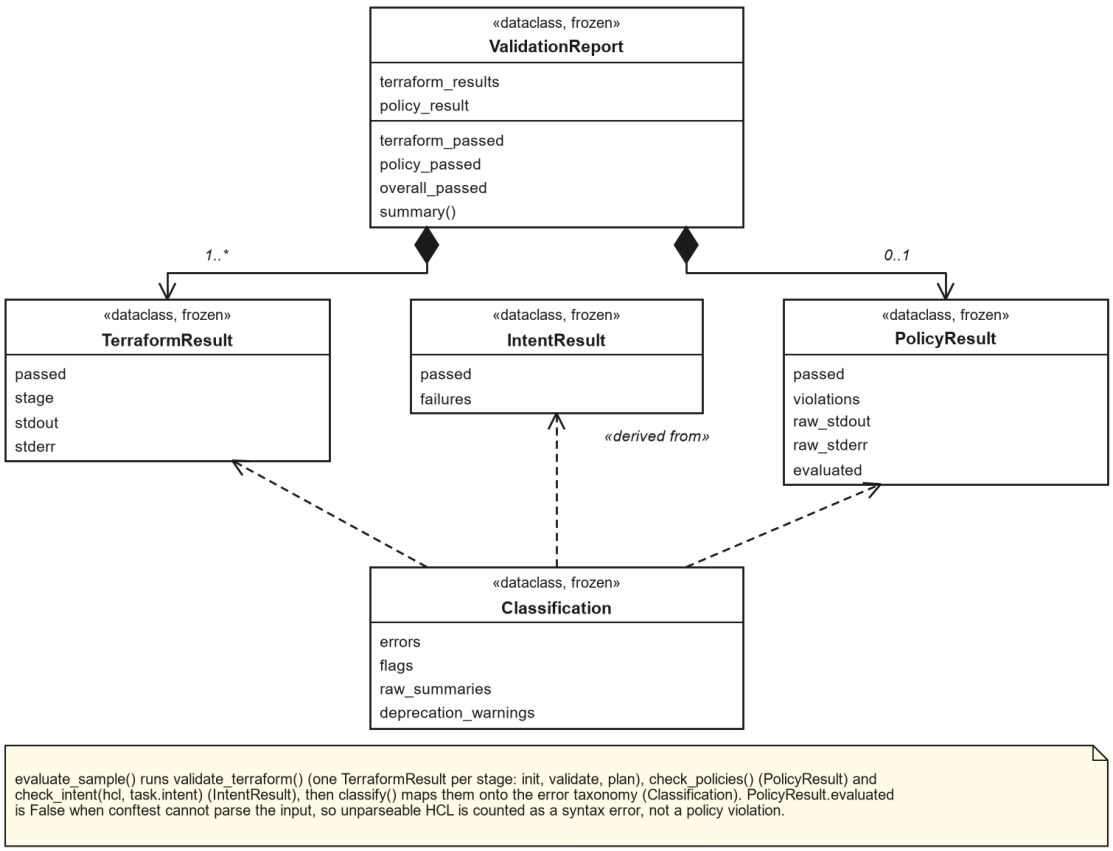


Figure 6: Class diagram of the validation and classification model.

3.9.4 Sequence view

Figure 7 follows one RAG request. The retriever embeds the requirement and asks the vector store for more candidates than needed (five times top-k) when a per-document cap is set, then applies the cap to choose the top-k chunks. The RagGenerator sends the prompt and context to Gemini and returns the Terraform together with the retrieved chunks. Validation is a separate step: the pipeline runs terraform init, validate and plan, then the conftest policy check, and returns a ValidationReport. Embedding and generation calls retry with exponential backoff on rate-limit and availability errors.

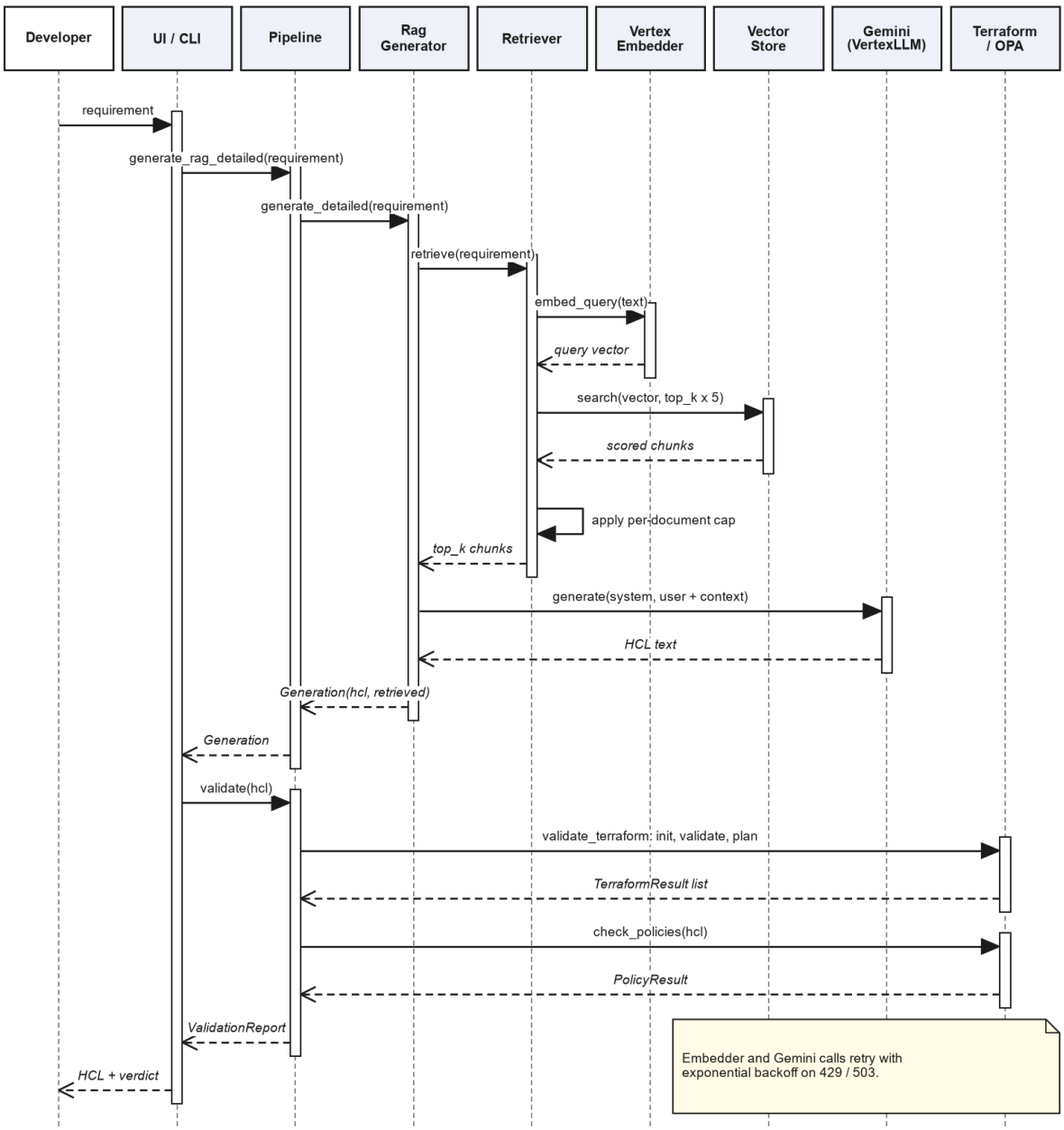


Figure 7: Sequence diagram of a RAG request and its validation.

3.9.5 Activity view

Figure 8 shows what the benchmark does for each sample. Samples already recorded are skipped, which makes a run resumable. The arm selects how the Terraform is generated; a failed generation is recorded as a result, not a crash. Terraform, policy and intent checks follow, and errors are classified. A sample lost to infrastructure failure is flagged and not scored, and is re-run later. When all samples are done, the run is summarised.



Figure 8: Activity diagram of the benchmark evaluation.

3.9.6 Deployment view

The prototype runs on one developer workstation: a Python environment with the pipeline and the Streamlit UI, terraform and conftest as local programs, and the knowledge base, vector index and results as files. It calls Vertex AI over HTTPS with Application Default Credentials for embeddings and generation. Terraform downloads the Google provider from the Terraform Registry and caches it. The provider

documentation was fetched once from GitHub. Terraform plan makes read-only calls to Google Cloud and never applies.

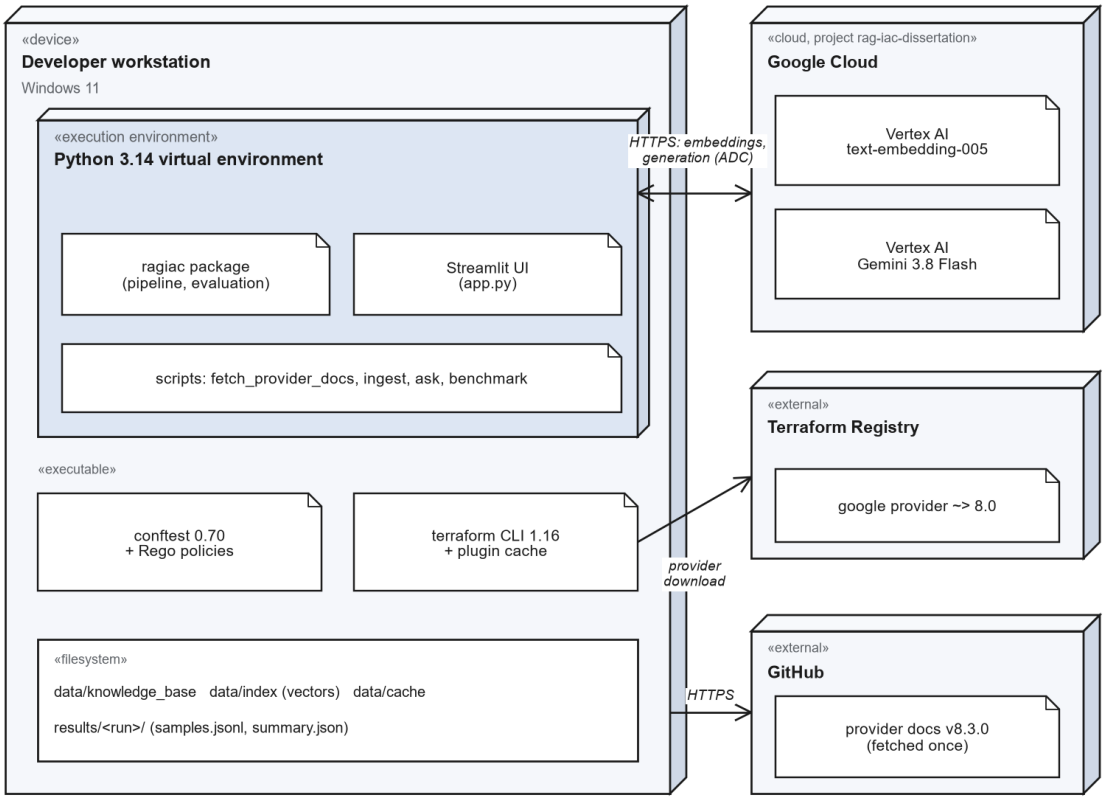


Figure 9: Deployment diagram of the prototype.

4. IMPLEMENTATION

4.1 Code base

The prototype is a Python package organised by pipeline stage: ingestion, embeddings, vector store, retrieval, generation, validation and evaluation. Every external dependency (Vertex AI, terraform, conftest) sits behind a small interface, which allows the full pipeline to run offline in a development mode and lets unit tests substitute fakes. A browser interface built with Streamlit lets a user submit a requirement and see the retrieved context, the generated Terraform and the validation results side by side, and browse stored benchmark runs. The code base has 105 automated tests, including integration tests that run the real policy engine.

A rule-based template generator exists so that the pipeline can be exercised without cloud credentials. It is not a language model, is clearly labelled as such in its output, and none of the results in this report come from it.

4.2 Reproducibility

Each benchmark run writes a manifest recording the git commit, model, sampling temperature, embedding model, chunking and retrieval settings, index size, provider-documentation release, target provider constraint, Terraform and Conftest versions, and hashes of the task file and the policy files. Every generated configuration, its retrieved chunks and the full validator diagnostics are stored per sample, and all summary statistics are computed from those stored records, so a table in this report can be regenerated or audited from the raw data. Runs are resumable, and samples lost to infrastructure failure can be re-run without touching the rest.

4.3 Local development mode

For offline development the pipeline substitutes a TF-IDF embedder, the in-memory vector store and the rule-based template generator. This mode is used for testing the plumbing only.

4.4 Engineering problems found and fixed

Building the evaluation infrastructure produced a number of defects, several of which would have silently distorted results had they not been found. They are recorded here because they show why the measurement tooling needed its own testing, and because they are candidate entries for the error analysis of tool-assisted generation.

Problem	Effect if unnoticed	Fix
Model reply wrapped in a Markdown fence	`terraform init` fails on every sample, so every sample would count as a syntax error	Extract the HCL from fenced replies, including replies with surrounding prose or a truncated closing fence
Newer model returns several content parts	Text accessor raises; samples lost or mis-scored	Join text parts, skip reasoning parts; found during the first full run
Model identifier retired (`gemini-2.0-flash`)	All generation fails with a 404	Moved to the current GA model and recorded the shutdown date of the alternative
Retry caught the wrong exception class for HTTP 429	No retries; ingestion and generation fail under quota limits	Catch the REST transport's exception; regression test added
Policy rules written for an older Rego syntax and ignoring the parser's list wrapping	One rule denied every resource, another never fired, so public resources would have passed	Rules rewritten and tested against known-bad and known-good configurations
Re-ingestion appended instead of replacing chunks	Duplicate chunks skew retrieval	Idempotent upsert by chunk identifier
Retrieval hit measured by filename	Under-reports retrieval quality, because several files describe one resource	Match on the resource types the retrieved text actually mentions
Variables without defaults counted as plan failures	Idiomatic code penalised for something plan cannot evaluate	Classified as not evaluable, alongside credential errors

Table 4: Defects found in the tooling and their remedies.

5. EVALUATION METHODOLOGY

The metrics, arms and analysis described in this section were fixed before the benchmark was run, as the supervisor required.

5.1 Research questions

6. **RQ1 (technical correctness).** Does retrieval increase the proportion of generated configurations that validate, plan and satisfy the stated intent?
7. **RQ2 (organisational policy).** Does retrieval increase conformance to organisational policy, which the requirement itself never states, and is any gain due to retrieval or simply to the conventions being available to the model?
8. **RQ3 (errors).** Which error categories dominate, and how does retrieval change their distribution?
9. **RQ4 (retrieval quality).** How often does retrieval surface documentation for the resources a task needs, and is that associated with success?

5.2 Task set

The published IaC benchmarks reviewed target AWS, so a GCP task set was built for this work. It contains 35 natural-language requirements: 33 headline tasks and 2 tasks that deliberately conflict with organisational policy. Difficulty is graded 1 to 3: level 1 is a single resource with one or two constraints (10 tasks), level 2 a single resource with several interacting constraints (12 tasks), and level 3 several related resources (13 tasks). Table 5 summarises the categories; the full list is in Appendix A.

Category	Tasks	Coverage
storage	6	Cloud Storage buckets: versioning, lifecycle rules, access settings, IAM
database	5	Cloud SQL: engine and tier, availability, backups, private IP, users and databases
kubernetes	4	GKE clusters and node pools: private nodes, Workload Identity, Autopilot, autoscaling
network	6	VPC, subnets, firewall rules, Cloud Router and NAT, global addresses, private DNS
compute	2	Compute Engine instances with specific machine, zone, image and service-account settings
serverless	3	Cloud Run, including IAM restricted to one invoker; Cloud Scheduler
messaging	2	Pub/Sub topics and subscriptions, including dead-letter policy
data	2	BigQuery datasets and tables
security	2	Secret Manager and Cloud KMS with rotation
platform	1	Artifact Registry
policy_conflict	2	Requirements that explicitly ask for something organisational policy forbids

Table 5: Task set composition.

Each task carries a machine-checkable intent specification in the style of IaC-Eval [2]: the resource types that must be present and assertions on their attributes (for example, that versioning is enabled and a lifecycle rule moves objects to NEARLINE after 30 days). Specifications allow alternatives where a requirement can legitimately be met in more than one way (an IAM member or an IAM binding; `node_count` or `initial_node_count`) and evaluate constant arithmetic, so a correct answer is not rejected for its form. The two policy-conflict tasks (a publicly readable website bucket and a Cloud SQL instance with a public IP) are reported separately: a model that does what was asked will violate policy by design, and mixing them into the headline would confuse the comparison.

5.3 Experimental arms, sampling and metrics

Three arms are compared. **RAG** supplies the eight retrieved chunks as context. The **baseline** uses the same model, target provider constraint and temperature but no context at all. A third, **conventions-in-prompt** control uses the RAG prompt with the four organisational-convention documents pasted in as context and no retrieval, and no provider documentation. The control exists because the conventions are stated only in the retrieval corpus: without it, any policy-compliance advantage of RAG over the baseline could simply reflect the model being told the conventions, not retrieval. RAG versus the control isolates the value of retrieving provider documentation; the control versus the baseline isolates the value of knowing the conventions.

Every task is run five times per arm at temperature 0.8, so that the samples differ and pass@k is meaningful; the sampling temperature is stored with each run. The run was interrupted once and resumed with identical settings and code. The first 93 RAG and baseline samples (roughly the first one to two samples of each task) come from the initial session on 19 September; every remaining sample, and all samples of the conventions-in-prompt control, were generated interleaved in the resumed session on 20 September. The control therefore has no samples from the earlier session, so drift in service behaviour between sessions could affect comparisons against it slightly. This is recorded in the run manifest, and the number of samples affected is small relative to the whole.

Metric	Definition
Validate pass rate	Proportion of samples for which `terraform init` and `terraform validate` succeed.
Plan pass rate	Proportion for which `terraform plan` succeeds, among samples where

Metric	Definition
	plan could be evaluated (credential and unset-variable failures excluded).
OPA violation rate	Proportion of samples that violate at least one policy rule, among samples the policy engine could parse.
Intent pass rate	Proportion of samples that satisfy the task's intent specification.
Functional pass rate	Validates, plans where evaluable, and satisfies intent: technical correctness independent of organisational policy.
Overall pass rate	Functional pass and no policy violation.
pass@k	Unbiased estimator [9] of the probability that at least one of k samples is overall-correct, averaged over tasks (k = 1, 3, 5).
Retrieval hit rate	Fraction of the resource types a task needs that appear in the retrieved chunks (RAG arm only).

Table 6: Metrics (validate/plan pass rate, OPA violation rate and pass@k were specified by the supervisor; the remainder were added).

Reporting functional and overall pass rates separately is deliberate. Following the observation in [6] that validity and security are largely orthogonal, and the intent-versus-correctness gap in [3], a single score would hide whether retrieval helps the model write correct infrastructure, helps it follow organisational conventions, or both.

5.4 Retrieval quality and the error taxonomy

Every failed sample is mapped to categories derived from what the validators report, so failures can be counted per arm rather than read one at a time. The categories are grouped by the stage that catches them: generation failure; provider initialisation; HCL syntax; schema errors (unsupported argument or block, missing required argument, invalid value, conflicting arguments, invalid reference, expression error); plan errors; policy violations (missing labels, public access); and intent failures (missing resource, attribute mismatch). Two informational flags are recorded that are not failures: use of deprecated arguments and a provider version constraint that differs from the target. Messages that fit no category fall into an *other* bucket with the raw diagnostic retained, so the taxonomy can be refined against real data and, in the final report, mapped onto the published taxonomy of Nekrasov et al. [3].

Retrieval quality is measured as the hit rate defined in Table 6: the share of needed resource types whose documentation appears among the retrieved chunks. Matching on content rather than file names is necessary because several documents can describe one resource.

5.5 Statistical treatment

Proportions are reported with 95% Wilson score intervals [10], which behave sensibly at small sample sizes and at 0% and 100%. Because the same 33 tasks are run under both arms, the arms are compared per task: for each task the mean overall pass rate of

each arm is computed, and the number of tasks on which each arm is better is tested with an exact two-sided sign test. With 33 tasks and one model, differences at the level of individual categories are indicative rather than conclusive and are described as such.

5.6 Threats to validity

- **Own task set and specifications.** Tasks and intent checks were written by the author. Specifications could wrongly reject a correct answer or accept an incorrect one. The specifications were revised after early trial runs to allow legitimate alternatives, so those trial tasks are not independent of the task design; the changes were made to the checker's tolerance, never to favour an arm.
- **Static intent checking.** Intent is checked on the parsed HCL. Values computed through variables or `for_each` cannot be resolved, so tasks were written to avoid needing that.
- **Asymmetry in organisational knowledge.** The organisational conventions live only in the retrieval corpus, so the baseline cannot know them. The policy-compliance advantage of RAG over the baseline is therefore partly an advantage of *having access to* those conventions, not of retrieval as such. The conventions-in-prompt control arm (Section 5.3) exists to separate the two. The conventions themselves were written by the author for this study, so their effect measures the value of stating conventions, not the value of any particular real organisation's documents.
- **Single model, single temperature.** Results are for one model at one sampling temperature and should not be generalised across models.
- **Model and service drift.** Hosted models change and are retired; the exact model, date and settings are recorded per run.
- **Sample size.** 33 tasks with five samples each gives usable but wide intervals; small differences will not be statistically distinguishable.
- **Plan evaluability.** Plan is only counted where it could run; excluding non-evaluable samples slightly shrinks the plan denominator.

6. RESULTS

6.1 The run

The reported run evaluates 33 headline tasks under 3 arms with 5 samples per task per arm (495 headline samples; 525 including the policy-conflict tasks), using `gemini-3.8-flash` at temperature 0.8, `terraform plan` enabled, and an index of 1034 chunks. 0 sample(s) remain unscored because of infrastructure failure; samples that failed for such reasons during the run were generated again (Section 7). Full settings are in Appendix D.

6.2 Headline metrics

Metric	Baseline	Conventions in prompt	RAG
Validate pass (init + validate)	99.4% [97-100]	97.6% [94-99]	98.8% [96-100]
Plan pass (where evaluable)	98.5% [95-100]	97.4% [93-99]	100.0% [98-100]
Intent pass	98.2% [95-99]	99.4% [97-100]	99.4% [97-100]
Functional pass (validate + plan + intent)	97.0% [93-99]	95.2% [91-98]	98.8% [96-100]
OPA violation rate (of evaluable)	42.7% [35-50]	0.0% [0-2]	20.1% [15-27]
Overall pass (functional + policy)	57.0% [49-64]	95.2% [91-98]	78.8% [72-84]
pass@1 (valid)	99.4%	97.6%	98.8%
pass@1 (overall)	57.0%	95.2%	78.8%
pass@3 (valid)	100.0%	100.0%	100.0%
pass@3 (overall)	57.6%	98.8%	83.3%
pass@5 (valid)	100.0%	100.0%	100.0%
pass@5 (overall)	57.6%	100.0%	84.8%
Retrieval hit rate	-	-	97.0%

Table 7: Headline metrics. Proportions show the 95% Wilson interval in brackets.

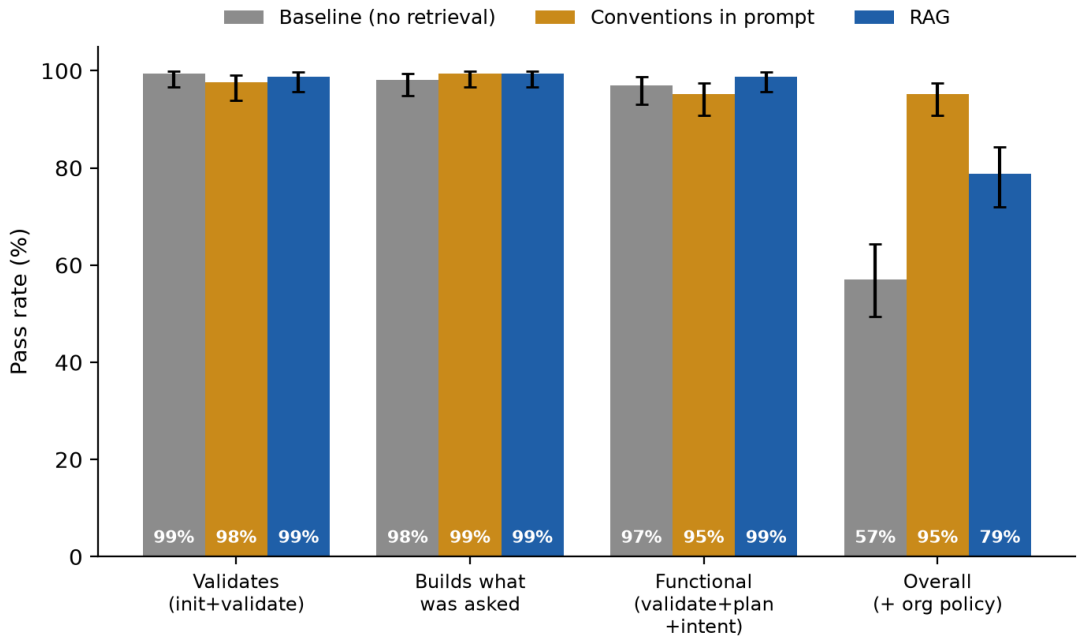


Figure 10: Pass rates by arm with 95% confidence intervals.

The arms are almost indistinguishable on technical correctness and clearly different on policy. Validate pass rates are 99.4% (baseline), 97.6% (control) and 98.8% (RAG); functional pass rates, which add plan and intent, are 97.0%, 95.2% and 98.8%. The intervals overlap almost completely, and each arm has only a handful of functional failures in 165 samples, so this experiment provides no evidence that retrieval improves technical correctness (RQ1). The model used is strong enough that the task set is close to its ceiling on this axis.

The overall pass rate, which also requires policy compliance, separates the arms: 57.0% for the baseline, 78.8% for RAG and 95.2% for the conventions-in-prompt control, with non-overlapping 95% intervals for the baseline against either of the others. The OPA violation rate tells the same story, at 42.7%, 20.1% and 0.0%. Sampling more does not repair the baseline: its pass@5 is 57.6%, essentially its pass@1 of 57.0%, which means its policy failures are systematic omissions and not bad luck on a given draw. RAG rises from 78.8% at pass@1 to 84.8% at pass@5, and the control from 95.2% to 100%.

6.3 Paired comparison across tasks

Comparing RAG with the baseline task by task, RAG had the higher overall pass rate on 10 of 33 tasks, the baseline on 0, and they tied on 23. The mean per-task difference

in overall pass rate is +21.8 percentage points and the exact two-sided sign test gives p = 0.0020.

Task	Requirement	RAG minus baseline
gcs-01	A Cloud Storage bucket named acme-invoices-archive in asia-south1 with object versioning enabled.	+40 pts
gcs-03	A Cloud Storage bucket with uniform bucket-level access enabled and public access prevention enforced.	+80 pts
gcs-06	A private Cloud Storage bucket and a service account that has read-only object access (roles/storage.objectViewer) to that bucket.	+80 pts
sql-01	A PostgreSQL 15 Cloud SQL instance using the db-custom-2-8192 machine tier.	+80 pts
gcs-02	A Cloud Storage bucket with a lifecycle rule that moves objects to NEARLINE storage after 30 days.	+100 pts
gcs-04	A Cloud Storage bucket in europe-west1 using the COLDLINE storage class.	+100 pts
gke-01	A regional GKE cluster in asia-south1 that removes the default node pool, plus a separately managed node pool of 3 e2-standard-4 nodes.	+100 pts
gke-03	A GKE Autopilot cluster in asia-south1.	+100 pts

Table 8: Tasks with the largest per-task differences between RAG and the baseline (at least 40 percentage points).

Comparing RAG with the conventions-in-prompt control task by task, RAG had the higher overall pass rate on 3 of 33 tasks, the conventions-in-prompt control on 10, and they tied on 20. The mean per-task difference in overall pass rate is -16.4 percentage points and the exact two-sided sign test gives p = 0.0923.

Task	Requirement	RAG minus conventions-in-prompt control
sql-03	A PostgreSQL Cloud SQL instance with no public IP address and deletion protection enabled.	-100 pts
sql-05	A Cloud SQL PostgreSQL instance with point-in-time recovery enabled.	-100 pts
gke-02	A private GKE cluster (private nodes) with Workload Identity enabled.	-100 pts
gcs-05	A Cloud Storage bucket that automatically deletes objects 90 days after creation.	-80 pts
sql-04	A PostgreSQL Cloud SQL instance with a database named appdb and a user named appuser.	-80 pts
gcs-01	A Cloud Storage bucket named acme-invoices-archive in asia-south1 with object versioning enabled.	-60 pts

Table 9: Tasks with the largest per-task differences between RAG and the conventions-in-prompt control (at least 40 percentage points).

The paired comparison, which treats each task as one observation and so does not depend on the number of samples, agrees with the pooled rates. RAG is never worse than the baseline on any task: it is better on 10 tasks and tied on the other 23, a mean gain of 21.8 percentage points (sign test p = 0.002). The tasks where it helps most are Cloud Storage, Cloud SQL and GKE tasks, all resource types that organisational policy requires to carry an environment label.

Against the control the direction reverses. The control is better on 10 tasks, RAG on 3, and 20 tie, a mean difference of 16.4 points in the control's favour. This is not significant at the 5% level in this run (p = 0.092), although the near-greedy run in

Section 6.9 gives a smaller p-value (p = 0.031) for the same direction. These two runs use the same 33 tasks and are not independent evidence, and several comparisons are being made, so the difference between RAG and the control should be read as a consistent trend and not as a settled result.

6.4 Error analysis

Error category	Baseline (of 165)	Conventions in prompt (of 165)	RAG (of 165)
POLICY_MISSING_LABELS	70 (42%)	0 (0%)	33 (20%)
POLICY_PUBLIC_ACCESS	19 (12%)	0 (0%)	0 (0%)
PROVIDER_INIT	1 (1%)	4 (2%)	1 (1%)
PLAN_ERROR	2 (1%)	4 (2%)	0 (0%)
INTENT_ATTRIBUTE_MISMATCH	3 (2%)	1 (1%)	1 (1%)
HCL_SYNTAX	1 (1%)	2 (1%)	1 (1%)
INVALID_REFERENCE	0 (0%)	0 (0%)	1 (1%)
INTENT_MISSING_RESOURCE	0 (0%)	0 (0%)	1 (1%)

Table 10: Samples containing each error category. A sample can contain several.

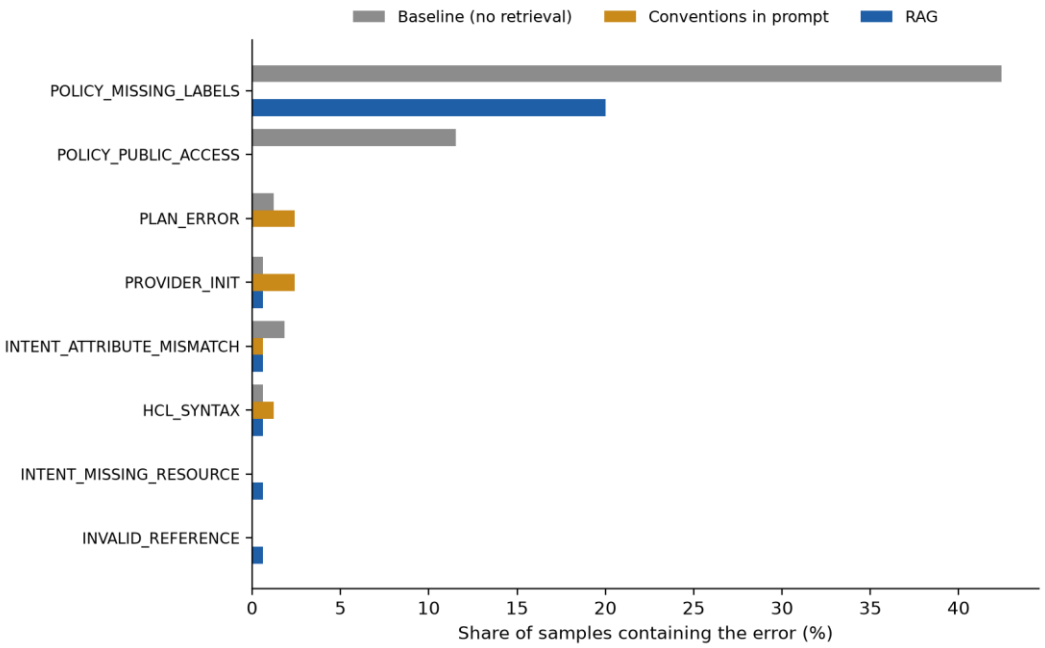


Figure 11: Share of samples containing each error category.

Most frequent non-policy diagnostics	Baseline	Conventions in prompt	RAG
plan: No value for required variable	40	8	4
plan: Failed to retrieve region, pid: , err: region: required field is not set	4	0	5
plan: point_in_time_recovery_enabled is only available for the following [POSTGRES SQLSERVER]. Y	2	4	0
init: Terraform encountered problems during initialisation, including problems	1	2	1
intent: google_container_node_pool.node_count | initial_node_count does not satisfy {'equals': 3}	2	0	0
intent: unparseable HCL: UnexpectedToken	1	1	0
init: Invalid multi-line string	0	1	1
init: Failed to install provider	0	2	0

Most frequent non-policy diagnostics	Baseline	Conventions in prompt	RAG
init: Argument or block definition required	1	0	0
init: Unterminated template string	0	1	0

Table 11: The most frequent validator and intent diagnostics (policy findings are shown in the previous table).

Informational flags (not counted as failures): Baseline: none; Conventions in prompt: PROVIDER_VERSION_MISMATCH in 2 samples; RAG: none.

Nearly all failures are organisational-policy failures, not technical ones. The dominant category is a missing environment label (POLICY_MISSING_LABELS), present in 42% of baseline samples, 20% of RAG samples and none of the control samples. Public access (POLICY_PUBLIC_ACCESS) appears in 12% of baseline samples and in no RAG or control sample. Every technical category (HCL syntax, invalid reference, plan error, provider initialisation, intent mismatch) is at most 2% in any arm, on counts of between 0 and 4 samples, which is too few to compare arms.

This answers RQ3: the errors that matter here are omissions of things the requirement never states, not malformed code. RAG removes the public-access errors entirely and halves the missing-label errors, but leaves one sample in five without the label. Two informational flags are worth noting: the control arm has two samples flagged for a provider version mismatch, which may be because its prompt contains example configurations written against a different provider version, though the data do not show that.

6.5 Retrieval quality and outcome

Retrieval	Samples	Functional pass	Overall pass
All needed resource docs retrieved (hit = 1)	155	99%	81%
Some needed docs missing (hit < 1)	10	90%	40%

Table 12: RAG-arm outcomes by whether documentation for every needed resource was retrieved.

Retrieval of the provider documentation works. The hit rate, the share of needed resource types whose documentation was retrieved, is 97.0%, so nearly every RAG sample was given the documentation for the resources it had to build. For this reason the two rows of the table above are very unequal: 155 of the 165 samples (94%) are in the first row.

Retrieval of the organisational conventions does not work, and this explains most of the gap to the control. The conventions document was retrieved in only 20 of 165 RAG samples (12%), and never for 29 of the 33 tasks. When it was retrieved, no sample was

missing a label and all 20 passed overall. When it was not, 33 of the remaining 145 samples were missing a label. Restricting to the three resource types the label rule covers (storage, database, Kubernetes, 75 samples), the label was missing in none of the 10 samples where the document was retrieved and in 33 of the 65 where it was not (51%). The likely reason is that retrieval matches the text of the requirement, which describes a resource and says nothing about labels, so a policy that applies to every resource is ranked below the resource-specific documentation.

Two cautions apply. The 20 samples where the document was retrieved come from only four tasks (bq-01, gke-01, gke-03 and net-02), so this is an association across few tasks and not proof of cause, and it was found by inspecting the results, not specified in advance. And the argument is about the conventions document specifically: the hit rate of 97% measures provider documentation, which is a different thing.

6.6 By category and difficulty

Category	Baseline	Conventions in prompt	RAG
compute	100%	100%	100%
data	90%	100%	100%
database	0%	80%	16%
kubernetes	25%	95%	75%
messaging	100%	90%	100%
network	100%	100%	100%
platform	100%	100%	100%
security	100%	90%	100%
serverless	100%	100%	100%
storage	0%	100%	70%

Table 13: Overall pass rate by task category.

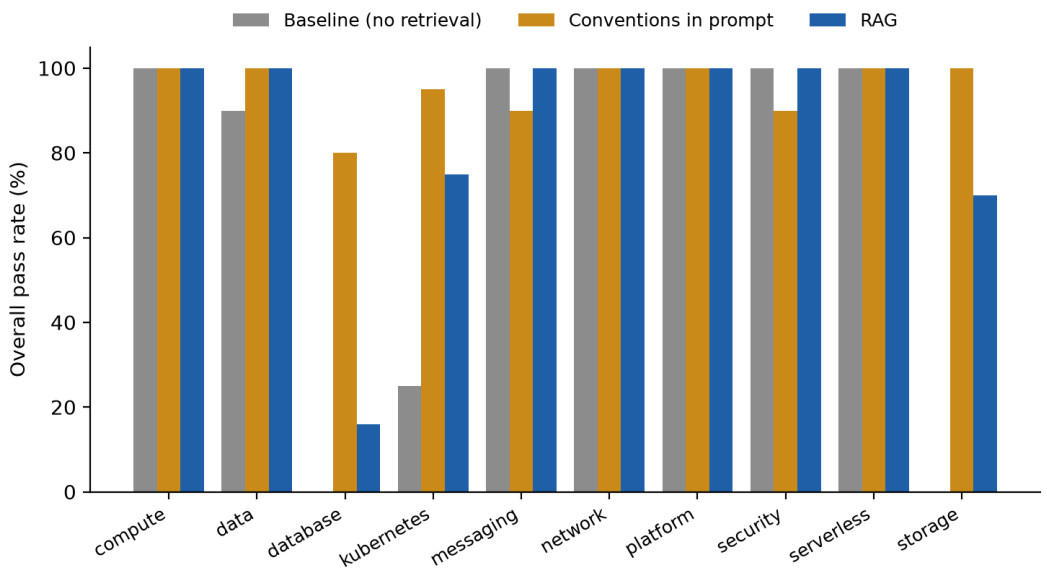


Figure 12: Overall pass rate by task category.

Difficulty	Baseline	Conventions in prompt	RAG
Level 1	50%	98%	84%
Level 2	40%	92%	58%
Level 3	75%	95%	91%

Table 14: Overall pass rate by difficulty level.

The pattern by category supports the same explanation. Compute, network, platform, serverless, messaging and security tasks pass at 90% or more in every arm, including the baseline, because no policy in this study applies to them or the model already complies. The differences sit entirely in the three resource types the label rule covers: storage (baseline 0%, RAG 70%, control 100%), database (0%, 16%, 80%) and Kubernetes (25%, 75%, 95%). Database is RAG's weakest category, and it is also the one where the control is furthest from ceiling, so it deserves a closer look in the error analysis.

By difficulty, the overall pass rate is not monotonic (RAG: 84% at level 1, 58% at level 2, 91% at level 3), and difficulty should not be read as the driver. Seven of the ten level-2 tasks are storage, database or Kubernetes tasks, the categories where the label rule bites, whereas only three of the thirteen level-3 tasks are. Difficulty and resource type are therefore confounded in this task set, and the category view is the more informative one.

6.7 Requirements that conflict with organisational policy

Metric	Baseline	Conventions in prompt	RAG
Intent pass (did what was asked)	100.0% [72-100]	100.0% [72-100]	100.0% [72-100]
OPA violation rate	100.0% [72-100]	100.0% [72-100]	100.0% [72-100]

Table 15: The two tasks that explicitly request policy-forbidden configurations.

When a requirement explicitly asks for something policy forbids, all three arms do what was asked: intent pass is 100% and the policy violation rate is 100% in every arm (10 samples each, two tasks with five samples). Neither retrieving nor pasting the conventions changed this, so no arm refused, adapted or warned. This is the behaviour a single-shot generator should be expected to show, and it is the reason the validation layer exists: the policy check is the only thing that stops a conflicting request. The sample is small (two tasks), so it is an observation and not a measurement of how often such conflicts are handled.

6.8 Retrieval ablations

Two retrieval settings were varied one at a time on the RAG arm, holding everything else fixed: the number of retrieved chunks (default 8, here 3) and the per-document cap (default 3 chunks per source document, here removed). The ablation runs use three samples per task rather than five, so pass@1 and pass@3 are the figures that can be compared across columns. Chunk size was not varied: doing so means rebuilding the index and re-embedding the corpus, which would overwrite the index the main run used (Section 8).

Metric	Default: 8 chunks, cap 3 (main run)	Top 3 chunks	No per-document cap
Samples	165	99	99
Validate pass	98.8% [96-100]	97.0% [91-99]	96.0% [90-98]
Intent pass	99.4% [97-100]	100.0% [96-100]	99.0% [94-100]
Overall pass	78.8% [72-84]	73.7% [64-81]	78.8% [70-86]
OPA violation rate (of evaluable)	20.1% [15-27]	23.5% [16-33]	19.6% [13-29]
pass@1 (overall)	78.8%	73.7%	78.8%
pass@3 (overall)	83.3%	78.8%	84.8%
Retrieval hit rate	97.0%	94.4%	97.0%

Table 16: RAG arm under different retrieval settings. Proportions show the 95% Wilson interval in brackets.

Reducing retrieval to three chunks lowers the overall pass rate slightly (73.7% against 78.8%) and the retrieval hit rate from 97.0% to 94.4%; removing the per-document cap leaves the overall pass rate unchanged (78.8%). The intervals overlap heavily in every comparison, so these runs do not show that either setting matters, and the default is not shown to be better than the alternatives.

The mechanism check from Section 6.5 was repeated for each run. The conventions document was retrieved in 12% of RAG samples with the default setting, 3% with three chunks and 9% without the cap, and in every case the share of samples missing a label stayed between 19% and 22%. Adjusting these two parameters within the range tried therefore does not fix the underlying problem, which is that the requirement text does not resemble the conventions text. Changes to the query, or to how conventions are supplied, are the more promising direction and were not tested. Chunk size was not varied (Section 8).

6.9 Near-greedy run (temperature 0)

To complement the sampled run, all arms were run once per task at temperature 0. This is a near-greedy setting, not a guarantee of determinism, because hosted models can still vary between calls. With one sample per task, pass@1 equals the pass rate.

Metric	Baseline	Conventions in prompt	RAG
Samples	33	33	33
Validate pass	97.0% [85-99]	97.0% [85-99]	100.0% [90-100]
Intent pass	97.0% [85-99]	97.0% [85-99]	100.0% [90-100]
Functional pass	97.0% [85-99]	97.0% [85-99]	100.0% [90-100]
Overall pass (pass@1)	57.6% [41-73]	97.0% [85-99]	78.8% [62-89]
OPA violation rate (of evaluable)	40.6% [26-58]	0.0% [0-11]	21.2% [11-38]

Table 17: Headline metrics at temperature 0, one sample per task.

At temperature 0 the ordering and the sizes of the gaps are the same as in the sampled run: overall pass is 57.6% for the baseline, 78.8% for RAG and 97.0% for the control, and RAG's functional pass is 100% against 97.0% for the others. With only 33 samples per arm the intervals are wide, and a difference of one or two samples in the functional metrics is within noise. Paired by task, RAG beats the baseline on 7 tasks and loses on none ($p = 0.016$), and the control beats RAG on 6 tasks and loses on none ($p = 0.031$). The agreement between the sampled and near-greedy runs indicates that the findings are not an artefact of sampling at temperature 0.8.

7. DISCUSSION

What retrieval did and did not do. Against the questions posed in Section 5.1: for RQ1 there is no measurable effect of retrieval on technical correctness, because every arm is near ceiling; for RQ2 retrieval clearly improves policy compliance over the baseline (overall pass 78.8% against 57.0%) but falls well short of simply stating the conventions in the prompt (95.2%); for RQ3 the errors that remain are almost all omissions of unstated policy such as labels and public access, not malformed code; and for RQ4 retrieval finds the provider documentation reliably (97.0%) but rarely finds the conventions (12% of samples), and the policy failures are concentrated where it does not. This is consistent with the observation in [6] that validity and security are largely independent: a model can write configuration that validates, plans and does what was asked, and still break organisational policy.

Why the control outperforms retrieval. The control is not an alternative to RAG so much as a measure of the ceiling of what the current conventions can deliver, and the gap between it and RAG is a retrieval-design problem. Policy is cross-cutting: it applies to a resource without being mentioned in the request, so ranking documents by similarity to the request ranks resource documentation first. Plausible remedies, none tested here, include always supplying a short policy summary, expanding the query with the resource types it names before retrieving, and indexing conventions per resource type. The control has a limit of its own: the four conventions documents are short enough to paste in whole, whereas a real organisation's conventions may not fit in a prompt, at which point retrieval, done well, becomes necessary. The experiment does not show that retrieval is unhelpful, only that with the present design and corpus it recovers about half of the available policy gain.

Threats to validity. These are set out in Section 5.6; the following are the ones that matter most for the reading of the results. The task set, the organisational conventions and the OPA policies were all written by the author, so compliance with the conventions partly measures whether the model reads and follows a short, clearly written document, and a real organisation's conventions would be messier. There are four policy rules and only three resource types are affected by the label rule, so the policy dimension is narrow. There are 33 headline tasks and one model, so intervals are wide and category-level results are indicative. The mechanism in Section 6.5 was found by inspecting the data and rests on four tasks. Finally, a number of samples failed for

reasons unrelated to the generated code, mainly terraform failing to load the provider plugin, dropped network connections and timeouts (11 in the main run, 5 in the top-3 ablation and 6 in the near-greedy run). These became more frequent once two runs shared the machine, and some occurred while it was asleep. These were never scored: each was flagged, removed and generated again, with the originals kept, so all reported figures come from completed evaluations. Several runs shared one machine and one API quota at times, which affects latency and the rate of such failures, not the content of the generated code.

Implications for the remaining work. The most useful next step is a targeted test of the query and corpus design suggested above, since that is where the measurable gap lies. Replacing the hand-written conventions with reviewed internal templates would test whether the result survives messier real-world text. The user evaluation asks a different question, whether developers trust and use the output, and depends on the approval described in Section 8.

8. REMAINING WORK AND PLAN

8.1 Remaining technical work

10. **Chunk-size ablation.** The number of retrieved chunks and the per-document cap were varied (Section 6.8), but chunk size was not, because it requires rebuilding and re-embedding the index. This remains to be done, into a separate index so the main run stays reproducible, and its effect related to retrieval hit rate (RQ4).
11. **Sanitised internal templates.** Replace the hand-written conventions with reviewed internal templates, containing no customer or production data, as the supervisor specified.
12. **Deeper error analysis.** Manually review a sample of failures per category, refine the taxonomy, and map it onto the published taxonomy [3].
13. **Optional extensions.** Verifier-feedback refinement in the style of [4], and a poisoning-robustness check of the corpus in the spirit of [7].

8.2 Proposed user evaluation

The outline calls for an evaluation of usability and developer trust. The supervisor asked that the participant count, task set and instrument be stated; the following is proposed and awaits his approval.

Element	Proposal
Participants	8 engineers from the delivery team, voluntary: 4 with under one year of Terraform experience and 4 with two or more years, to see whether the assistant helps novices and experts differently.
Design	Within-subjects and counterbalanced. Each participant completes four tasks matched for difficulty: two with the assistant and two without it (using provider documentation and search as they normally would). Task order and condition are rotated across participants.
Task set	Four synthetic tasks per participant drawn from the benchmark task set, spanning difficulty levels 1 to 3. No customer or production information is used.
Objective measures	Task success (the same automated validity and intent checks used in the benchmark), time to completion, and the number of edits made to assistant output.
Subjective instruments	The System Usability Scale (10 items) [13] after the assistant condition; a trust-in-automation scale (12 items) [14], adapted to code generation; and five semi-structured interview questions on what participants did and did not trust.
Analysis	Paired non-parametric comparisons (Wilcoxon signed-rank) given the small sample, with effect sizes and intervals rather than p-values alone; SUS scored in the standard way; interview notes coded thematically.
Ethics	Informed consent, anonymised data, organisational permission obtained in advance, and the option to withdraw at any time.

Table 18: Proposed user-evaluation design.

8.3 Future Plan

The phases below are those of the approved outline. Status is as at the date of this report.

Sl No	Phases	Start Date - End Date	Work to be done	Status
1	Literature review and problem formulation	25 Jul 2026 - 14 Aug 2026	Review of LLM code generation, RAG and IaC benchmarks; formulate the problem	COMPLETED
2	Abstract / outline submission and evaluation	15 Aug 2026 - 04 Sep 2026	Prepare and submit the dissertation outline	COMPLETED
3	RAG pipeline and knowledge-base design	05 Sep 2026 - 25 Sep 2026	Choose cloud, LLM, embedding model and vector store; design ingestion, retrieval, generation and validation (Section 3.3)	COMPLETED
4	Prototype development and mid-semester evaluation	26 Sep 2026 - 16 Oct 2026	Working prototype on Vertex AI, first full benchmark, mid-semester report	COMPLETED
5	Full implementation and validation tooling	17 Oct 2026 - 06 Nov 2026	Done ahead of schedule: validation tooling, control arm, retrieval-count and per-document-cap ablations, near-greedy run. Remaining: chunk-size ablation, reviewed internal templates, manual error analysis	IN PROGRESS
6	Benchmarking, user evaluation and final report	07 Nov 2026 - 30 Nov 2026	Final benchmark, user study (Section 8.2), final report	PENDING

Table 19: Plan of work and status.

8.4 Risks

- **Model retirement.** Hosted models are retired on short notice. Mitigated by recording model and date per run and by keeping the model a configuration setting.
- **API quota.** A new cloud project has low default quotas and long benchmark runs are throttled. Mitigated by backoff, an embedding cache and resumable runs; a quota-increase request is an option if needed.
- **Statistical power.** Wide intervals may limit conclusions. Mitigated by paired analysis and by reporting uncertainty explicitly.
- **Participant availability.** The user study depends on colleagues' time. Mitigated by a short, fixed-length session and a small design.

9. CONCLUSION

This work built a RAG assistant that generates and validates Terraform for Google Cloud, and evaluated it against a no-retrieval baseline and a conventions-in-prompt control on 35 tasks, three arms and five samples per task. The prototype works end to end on real cloud services, and the evaluation harness records every sample so that results can be reproduced and inspected.

The main findings are these. The model writes technically correct Terraform with or without retrieval, so retrieval brings no measurable gain in correctness on this task set. Retrieval does improve compliance with organisational policy, from 57.0% to 78.8% overall pass, and it never made a task worse than the baseline. But pasting the conventions into the prompt does better still (95.2%). The stored records suggest the reason: the conventions were retrieved in only 12% of samples, and the policy failures fall where they were absent. Retrieval of provider documentation works well, and retrieval of cross-cutting policy does not, with the present design. Tuning the number of chunks or the per-document cap did not change this, and a near-greedy run gave the same picture.

For an organisation, the practical lesson is that the value of a RAG assistant for IaC depends less on retrieving more documentation than on making sure policy reaches the model, and that automated policy validation remains essential, since every arm violated policy when the request itself asked for it. The remaining work is a chunk-size ablation, a test of query and corpus design that would surface the conventions, reviewed internal templates in place of the hand-written ones, deeper manual error analysis and the proposed user evaluation.

ABBREVIATIONS

Abbreviation	Expansion
ADC	Application Default Credentials
API	Application Programming Interface
CI	Confidence Interval
GCP	Google Cloud Platform
GCS	Google Cloud Storage
GKE	Google Kubernetes Engine
HCL	HashiCorp Configuration Language
IaC	Infrastructure as Code
KB	Knowledge Base
LLM	Large Language Model
OPA	Open Policy Agent
RAG	Retrieval-Augmented Generation
Rego	The policy language used by Open Policy Agent
RQ	Research Question
SUS	System Usability Scale
TF-IDF	Term Frequency - Inverse Document Frequency
WILP	Work Integrated Learning Programmes

Table 20: Abbreviations used in this report.